İSTANBUL ÜNİVERSİTESİ-CERRAHPAŞA

Bilgisayar Mühendisliği Bölümü

Veri Yapıları

ÖDEV - 2

Kargo Dağıtım Merkezi Simülasyonu - Stack ve Queue Uygulaması

1. Ödevin Amacı

Bu ödevin amacı, Stack (Yığın) ve Queue (Kuyruk) veri yapılarının gerçek bir problem üzerinde nasıl kullanılabileceğini anlamak ve bu veri yapılarını Python programlama dili ile implemente etmektir. Ödev kapsamında bir kargo dağıtım merkezinin kamyon ve paket yönetim sistemi geliştirilecektir.

2. Problem Tanımı

Bir kargo dağıtım merkezinde yalnızca bir adet yükleme rampası bulunmaktadır. Merkeze gelen kamyonlar, yükleme rampasını kullanabilmek için sıraya girmektedir. Kamyonların geliş sırası korunmalıdır.

- İlk gelen kamyon, yükleme rampasına ilk alınmalıdır.
- Yeni gelen kamyon, bekleme sırasının sonuna eklenmelidir.

Bir kamyon yükleme rampasına alındığında, kamyona paket yüklenebilmektedir. Paketler kamyon içerisinde üst üste yerleştirilmektedir. Bu nedenle:

- En son yüklenen paket ilk indirilebilir.
- Bir paketin altında bulunan başka bir pakete doğrudan erişilemez.

Bu sistemi uygun Stack ve Queue veri yapılarını kullanarak geliştiriniz.

3. Stack Veri Yapısı

Paketlerin yönetimi için bir Stack sınıfı oluşturulmalıdır. Stack sınıfı en az aşağıdaki metotları içermelidir:

```
push(item)
pop()
peek()
is_empty()
size()
```

push(item): Stack üzerine yeni bir eleman ekler.

pop(): Stack'in en üstündeki elemanı çıkarır ve döndürür.

peek(): Stack'in en üstündeki elemanı silmeden döndürür.

is_empty(): Stack boş ise True, değilse False döndürür.

size(): Stack içerisindeki eleman sayısını döndürür.

Paketlerin yönetimi LIFO (Last In - First Out) prensibine uygun olmalıdır.

4. Queue Veri Yapısı

Kamyonların bekleme sırası için bir Queue sınıfı oluşturulmalıdır. Queue sınıfı en az aşağıdaki metotları içermelidir:

```
enqueue(item)
dequeue()
peek()
is_empty()
size()
```

enqueue(item): Kuyruğun sonuna yeni bir eleman ekler.

dequeue(): Kuyruğun başındaki elemanı çıkarır ve döndürür.

peek(): Kuyruğun başındaki elemanı silmeden döndürür.

is_empty(): Queue boş ise True, değilse False döndürür.

size(): Kuyrukta bekleyen eleman sayısını döndürür.

Kamyonların yönetimi FIFO (First In - First Out) prensibine uygun olmalıdır.

5. Truck Sınıfı

Her kamyon bir nesne olarak temsil edilmelidir. Bir kamyon en az aşağıdaki bilgileri içermelidir:

```
plate
packages
capacity
```

Her kamyonun paketleri kendi Stack yapısı içerisinde tutulmalıdır. Her kamyonun maksimum kapasitesi 5 paket olarak kabul edilecektir.

6. Programda Bulunması Gereken İşlemler

6.1. Yeni Kamyon Ekleme

Kullanıcıdan kamyon plakası alınmalıdır. Yeni kamyon, bekleme kuyruğunun sonuna eklenmelidir.

```
Kamyon plakası: 34 ABC 123
34 ABC 123 kuyruğa eklendi.
```

6.2. Sıradaki Kamyonu Rampaya Alma

Bekleme kuyruğunun başındaki kamyon yükleme rampasına alınmalıdır. Rampada zaten bir kamyon bulunuyorsa ikinci bir kamyon rampaya alınmamalıdır.

```
34 ABC 123 yükleme rampasına alındı.
HATA: Yükleme rampası şu anda dolu.
```

6.3. Kamyona Paket Yükleme

Rampada bulunan kamyona paket yüklenebilmelidir. Her paket benzersiz bir paket koduna sahip olacaktır. Paketler Stack yapısına eklenmelidir.

```
Paket kodu: PKG001
PKG001 kamyona yüklendi.
```

```
TOP
 |
PKG002
PKG001
```

Kamyon kapasitesi dolduğunda yeni paket kabul edilmemelidir.

```
HATA: Kamyonun paket kapasitesi dolu.
```

6.4. Kamyondan Paket İndirme

Kullanıcı paket indirme işlemini seçtiğinde kamyona en son yüklenen paket indirilmelidir. Kamyonda paket bulunmuyorsa uygun hata mesajı gösterilmelidir.

```
PKG002 kamyondan indirildi.
HATA: Kamyonda indirilecek paket bulunmamaktadır.
```

6.5. Kamyonu Rampadan Çıkarma

Rampadaki kamyon işlemleri tamamlandığında merkezden ayrılabilmelidir. Rampanın boşalmasından sonra bekleme kuyruğundaki bir sonraki kamyon rampaya alınabilir.

```
34 ABC 123 merkezden ayrıldı.
```

6.6. Bekleyen Kamyonları Görüntüleme

Kamyonlar geliş sırasına göre görüntülenmelidir.

```
Bekleyen Kamyonlar


1. 06 XYZ 55
2. 35 KLM 42
3. 16 ABC 90
```

6.7. Rampadaki Kamyonu Görüntüleme

Rampada bulunan kamyonun plakası, taşıdığı paketler ve kapasite durumu görüntülenmelidir.

```
Rampadaki Kamyon: 34 ABC 123


Paketler:


TOP
 |
PKG004
PKG003
PKG002
PKG001


Kapasite: 4 / 5
```

7. Kullanıcı Menüsü

Program, kullanıcı çıkış seçeneğini seçene kadar çalışmaya devam etmelidir. Örnek menü:

```
================================
     KARGO DAĞITIM MERKEZİ
================================


1 - Yeni kamyon ekle
2 - Sıradaki kamyonu rampaya al
3 - Kamyona paket yükle
4 - Kamyondan paket indir
5 - Kamyonu rampadan çıkar
6 - Bekleyen kamyonları göster
7 - Rampadaki kamyonu göster
0 - Programdan çık


Seçiminiz:
```

8. Zorunlu Kurallar

1. Stack ve Queue veri yapıları öğrenci tarafından implemente edilmelidir.
2. Python'ın hazır Queue yapıları (ör. queue.Queue, collections.deque) kullanılmamalıdır.
3. Paketlerin yönetiminde Stack kullanılmalıdır.
4. Kamyonların bekleme sırasında Queue kullanılmalıdır.
5. Stack ve Queue sınıfları birbirinden bağımsız olarak tasarlanmalıdır.
6. Program hatalı işlemlerde çökmemeli ve kullanıcıya uygun hata mesajı vermelidir.

Aşağıdaki durumlar mutlaka kontrol edilmelidir:

- Boş Stack'ten pop yapılması
- Boş Queue'dan dequeue yapılması
- Rampada kamyon yokken paket yüklenmesi
- Rampada kamyon yokken paket indirilmesi
- Rampada kamyon varken ikinci kamyonun alınması
- Kapasitesi dolu kamyona paket yüklenmesi

9. Örnek Program Akışı

```
> 1
Kamyon plakası: 34 ABC 123
34 ABC 123 kuyruğa eklendi.


> 1
Kamyon plakası: 06 XYZ 55
06 XYZ 55 kuyruğa eklendi.
```

```
> 1
Kamyon plakası: 35 DEF 78
35 DEF 78 kuyruğa eklendi.

> 6
Bekleyen Kamyonlar:
1. 34 ABC 123
2. 06 XYZ 55
3. 35 DEF 78

> 2
34 ABC 123 yükleme rampasına alındı.

> 3
Paket kodu: PKG001
PKG001 kamyona yüklendi.

> 3
Paket kodu: PKG002
PKG002 kamyona yüklendi.

> 4
PKG002 kamyondan indirildi.

> 5
34 ABC 123 merkezden ayrıldı.
```

10. Bonus: Acil Paket Arama

Bir müşteri belirli bir paketin acil olarak kamyondan çıkarılmasını isteyebilir. Örneğin kamyonun mevcut durumu aşağıdaki gibi olsun:

```
TOP
 |
PKG005
PKG004
PKG003
PKG002
PKG001
```

Kullanıcı PKG002 paketini aradığında program, bu pakete ulaşmak için kaç paketin önce çıkarılması gerektiğini hesaplamalıdır.

```
PKG002 paketine ulaşmak için 3 paket çıkarılmalıdır.
```

Önemli: Stack yapısının çalışma prensibi bozulmamalıdır. Aranan pakete ulaşmak amacıyla Stack içerisindeki elemanlara indeks kullanılarak doğrudan erişim yapılmamalıdır.

11. Rapor Soruları

1. Kamyonların bekleme sırasında neden Stack yerine Queue kullanılmıştır?

2. Paketlerin yönetiminde neden Queue yerine Stack kullanılmıştır?

3. Stack yapısındaki push ve pop işlemlerinin zaman karmaşıklığı nedir?

4. Queue yapısındaki enqueue ve dequeue işlemlerinin zaman karmaşıklığı nedir?

5. Stack'in en altında bulunan bir elemana erişmenin neden zor olduğunu açıklayınız.

6. Stack ve Queue yapılarının Python listeleri kullanılarak gerçekleştirilmesi mümkün müdür? Veri yapılarını ayrı sınıflar olarak tasarlamanın avantajlarını açıklayınız.

12. Değerlendirme

Bölüm	Puan
Stack implementasyonu	20
Queue implementasyonu	20
Truck sınıfı	10
Kargo merkezi işlemleri	20
Stack ve Queue'nun doğru kullanılması	10
Hata durumlarının yönetilmesi	10
Kod düzeni ve okunabilirlik	5
Rapor soruları	5
Toplam	**100**

Bonus: Acil paket arama özelliğinin doğru gerçekleştirilmesi +10 puan.

13. Teslim Formatı

Teslim edilecek dosyalar:

```
ogrenciNo_ad_soyad.zip içerisinde
odev2.py
odev2_rapor.pdf
```

Kaynak kod içerisinde gerekli açıklamalar ve yorum satırları bulunmalıdır. Öğrenci, teslim ettiği kod içerisindeki sınıfları, metotları ve kullanılan veri yapılarının çalışma mantığını açıklayabilmelidir.

Gerekli görüldüğü durumda öğrenciye teslim ettiği kod ile ilgili sözlü sorular yöneltilebilir.